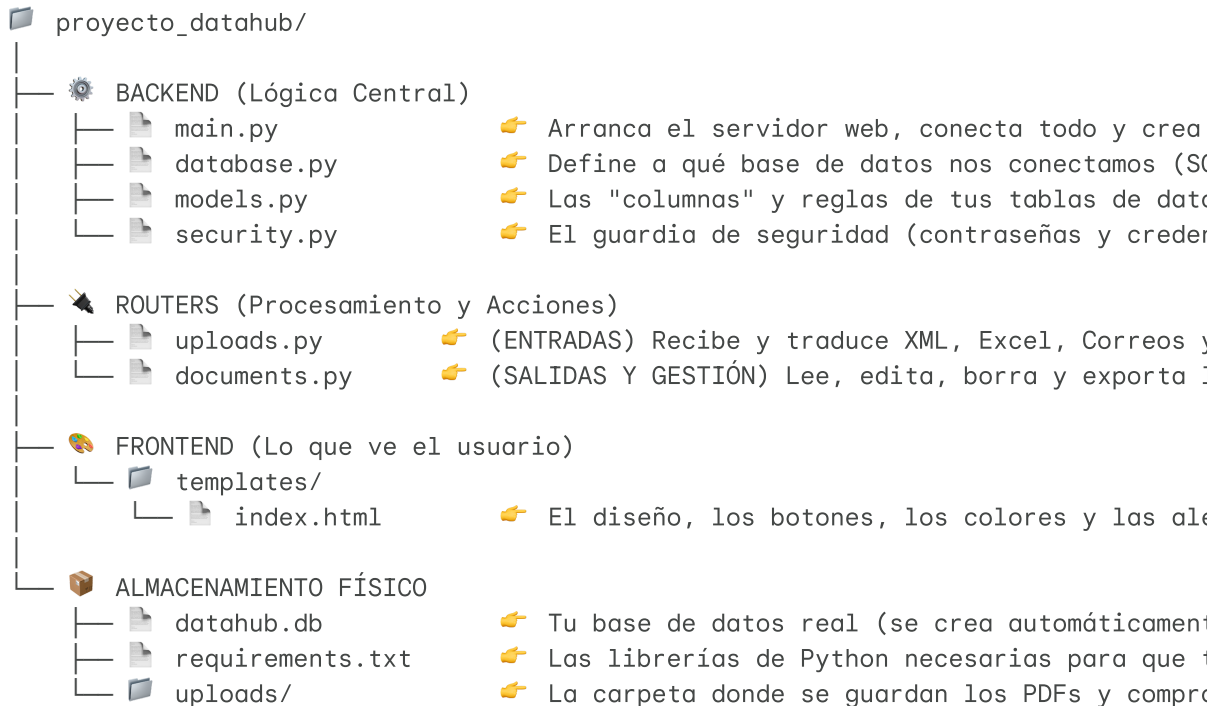


Guía de Arquitectura y Modificaciones: DataHub ULMA

Este esquema es tu manual de control definitivo. Aquí encontrarás la función exacta de cada archivo y qué puedes modificar dentro de él según tus necesidades (estilos, lógicas, permisos, entradas y salidas).

ESQUEMA GENERAL DEL PROYECTO



MANUAL DE MODIFICACIONES POR ARCHIVO

1. templates/index.html (La Interfaz Visual)

Función: Es la pantalla de tu sistema. Controla todo lo visual, cómo interactúa el usuario y manda las peticiones al backend. **¿Qué puedes modificar aquí?**

- **Títulos y Textos:** Puedes cambiar las palabras de la interfaz (ej. cambiar `<title>Portal DataHub</title>` o `<h2>REGISTROS CONSOLIDADOS</h2>`).
- **Tamaños y Colores:** Modificar el CSS de Tailwind. Por ejemplo, si quieres un botón más ancho usas `w-full`, o si quieres cambiar el rojo corporativo, editas la clase `.ulma-red { background-color: #E3000F; }`.

- **Lógica de Filtros Visuales:** En la función de JavaScript `applyFilters()` , puedes cambiar cómo busca la barra (ej. agregar que busque por "Estado de Pago" además de RFC).
- **Formato de Moneda y Fechas:** En `row.total.toLocaleString('es-MX')` puedes cambiar si quieres que lleve signo de dólar o euros, y la cantidad de decimales visualizados.
- **Alertas:** Cambiar textos de confirmación como `confirm("¿Seguro que deseas eliminar este registro para siempre?")` .

2. `models.py` (La Estructura de Datos)

Función: Define exactamente qué información se guarda en la base de datos. **¿Qué puedes modificar aquí?**

- **Nuevos Campos/Entradas:** Si en el futuro requieres agregar el "Departamento" al registro, debes añadir aquí: `departamento = Column(String, default="Administración")` .
- **Reglas por Defecto:** Puedes cambiar qué pasa si un dato viene vacío. Por ejemplo, en `estado_pago = Column(String, default="Pendiente")` , podrías cambiar el valor inicial a "En Revisión".
- **Formatos de Datos:** Elegir si un dato es un número entero (`Integer`), un texto (`String`), un decimal (`Float`) o una fecha (`DateTime`).

3. `routers/uploads.py` (Lógicas de Entrada y Extracción)

Función: Procesa todo lo que el usuario "Sube" (entradas de datos). **¿Qué puedes modificar aquí?**

- **Reglas de Extracción de Textos (Regex):** En la ruta `/subir-texto` , si los correos que te llegan cambian su formato, aquí modificas las reglas `re.search` para decirle a Python cómo encontrar el RFC o la cantidad de dinero exacta.
- **Formatos de Excel Aceptados:** En `/subir-excel` , aquí defines qué nombres de columnas espera tu sistema (ej. `row.get('RFC')` o `row.get('Total')`). Si tu formato de Excel cambia la columna a "Monto_Total", aquí lo ajustas.
- **Lógicas de XML:** En `/subir-xml` , aquí le indicas a Python qué etiquetas del archivo XML te importan. Actualmente lee `Total` , `Rfc` , `Nombre` y `UUID` . Puedes agregar más si la factura trae otros datos fiscales relevantes.

4. `routers/documents.py` (Lógicas de Salida, Permisos y Gestión)

Función: Maneja lo que pasa con los datos que ya están adentro (exportar, editar, borrar). **¿Qué puedes modificar aquí?**

- **Privilegios y Permisos:** Aquí decides qué botones funcionan para quién. Por ejemplo, la línea `if current_user.role != "admin": raise HTTPException(...)` bloquea la descarga de Excel para usuarios normales. Puedes borrar esta regla si quieres que todos puedan descargar el Excel.
- **Salidas de Datos (Formatos):** En `/descargar-excel`, aquí construyes cómo sale tu reporte. Puedes agregar, quitar o cambiar el orden de las columnas en el Excel final modificando la variable `data = [{"ID": d.id, "Origen": d.tipo...}]`.
- **Reglas de Edición:** En la ruta `put("/documentos/{doc_id}")`, aquí se listan los ÚNICOS campos que el usuario tiene permitido modificar (actualmente solo Centro de Costo y Fecha de Pago). Si quieres que también puedan editar el Nombre, debes añadir esa regla aquí.

5. `security.py` (Autenticación)

Función: Protege el sistema mediante cifrado. **¿Qué puedes modificar aquí?**

- **Formato de Hashes:** Aquí se define que las contraseñas se encripten usando `bcrypt` y `gensalt`.
- **Reglas de Expiración:** (Si en el futuro deseas agregarlas) Aquí puedes definir cuánto dura una sesión abierta antes de pedir la contraseña nuevamente.

6. `main.py` (Inicialización y Variables Globales)

Función: Es el "Switch de encendido". Ensambla los routers, la seguridad y el frontend. **¿Qué puedes modificar aquí?**

- **Usuarios Iniciales:** En la función `startup_event()`, aquí se crean los usuarios por defecto cuando se borra la BD. Puedes cambiar las contraseñas base o agregar usuarios como `auditor` con sus propias contraseñas.
- **Reglas de Acceso Externo (CORS):** En `CORSMiddleware`, puedes restringir qué computadoras o dominios pueden acceder a tu API.
- **Versión y Título de tu Web Service:** `app = FastAPI(title="DataHub Ulma", version="4.4")`.

7. `database.py` (El Motor Físico)

Función: Crea el puente con tu archivo SQLite. **¿Qué puedes modificar aquí?**

- **Tipo de Base de Datos:** Si en 2 años el DataHub crece tanto que SQLite no es suficiente, en este único archivo cambias la cadena de texto de la URL para conectarte a un PostgreSQL o un MySQL corporativo en la nube. ¡v el resto de los archivos funcionarán